

Bridge Day 13 STUDENT WORKBOOK

Loop Design, Break, Continue, and Nested Tracing

Learning sequence: Predict - Trace - Explain - Test - Interpret

Wednesday, July 22, 2026 • 50 points • longitudinal template 07242026_v1

Name		UIN		Section	
------	--	-----	--	---------	--

Daily target and independence boundary

Select skip, stop, and reset behavior while tracing repeated work inside repeated work.

Allowed tools	Prior tools plus break, continue, and nested for-loop tracing
Support supplied	Behavioral requirements and a sample input stream are supplied.
Student decides	Loop form, placement of skip and stop decisions, state updates, and reset scope.
Scaffold level	Requirements only; students select the control-flow pattern.

Evidence and scoring

Evidence	Points	Secure work shows
Integrated trace	10	intermediate state, condition results, exact output, and meaning
Diagnosis and repair	8	actual, intended, cause, repair, and a discriminating test
Complete program and tests	32	coherent executable logic, required output, assumptions, and defended tests

Task 1. Integrated trace - 10 points

Trace nested repetition while distinguishing a skipped pass, an early stop, and a per-station reset.

```
overall = 0

for station in range(1, 3):
    station_total = 0
    for reading in [0, station + 1, 5]:
        if reading == 0:
            continue
        if reading == 5:
            break
        station_total = station_total + reading
    overall = overall + station_total
    print(station, station_total)

print(overall)
```

Your task

1. Give a complete reached-pass ledger that identifies the continue, break, station_total reset, and overall update.
2. Write every output line and explain why the inner break does not end the outer loop.

Task 2. Diagnose and repair - 8 points

The intent is to print every odd integer through 5 and then stop. The current order omits 5.

```
value = 0
while value < 8:
    value = value + 1
    if value % 2 == 0:
        continue
    if value == 5:
        break
    print(value)
```

Your task

1. Give actual versus intended output, identify the ordering cause, and write a minimal repair that still stops at 5.

Task 3. Construct a complete program - 32 points

Program specification

- Repeatedly read an integer inspection code.
- Code 0 ends normal input. A negative code is ignored and processing continues.
- A code of 90 or greater is a shutdown code: stop immediately without counting or adding it.
- Each positive code below 90 is accepted. Count accepted codes and add their values.
- After the loop, print `accepted_count` and `total` on separate lines.

Program workspace

Task 3 continued. Test and defend - included in 32 points

Continue code if needed. Required tests, expected results, and brief defense

Bridge Day 13 STUDENT WORKBOOK

VIP Evidence Handoff

Learning sequence: Predict - Trace - Explain - Test - Interpret

Contract 07a04826c975 • longitudinal template 07242026_v1

Why engineers use this page

Select a loop form from the requirement, complete a sample/transfer pair, then finish the visibly labeled Independent Program Studio whose final build is a complete input-to-output loop program.

Decision boundary: Code is useful only when its stored state, visible result, assumptions, units, limits, and tests support a defensible engineering interpretation.

Construct readiness before independent work

New authoring tools: for loop, while loop, range call, loop state update, termination condition, list literal as collector, append call

Carried authoring tools: assignment, print call, arithmetic expression, modulo, input call, int conversion, f string, comparison expression, if elif else

Supplied-code exposure only: list index read

An exposure-only construct may be traced in complete supplied code, but the independent task will not require students to invent it before its authoring day.

Notebook cells and task constructs

Activity	Work	Stable cells	Required authoring constructs
18.18	Multiples In Range	18-18-work / 18-18-transfer / 18-18-cold	append call, arithmetic expression, for loop, range call
18.19	Countdown To Multiple	18-19-work / 18-19-transfer / 18-19-cold	arithmetic expression, while loop
18.20	Smallest Value	18-20-work / 18-20-transfer / 18-20-cold	comparison expression, for loop, if elif else
18.21	Cooldown Sequence	18-21-work / 18-21-transfer / 18-21-cold	append call, arithmetic expression, while loop
18.22	Rounds To Clear Queue	18-22-work / 18-22-transfer / 18-22-cold	arithmetic expression, f string, input call, int conversion, while loop

Readiness evidence

1. Classify each activity as fixed traversal or condition-controlled repetition and justify one classification.
2. Choose one activity and list the complete state that must still be correct after the loop ends.

Timing and help trigger

Record a first prediction before running. If you still lack an executable plan after about 8 focused minutes, use the linked ThinkCSPy refresher or the supplied model. If two attempts fail for the same named requirement, write the exact mismatch and ask a peer mentor or instructor a targeted question. Timing is diagnostic evidence, not a speed grade. **Start or elapsed minutes:** _____

My help trigger:

Engineering Evidence Record

Complete one record from a model, transfer, or Independent Program Studio build. Generic statements are not sufficient; use actual values, a real revision, and one concrete next test.

Evidence field	Record
Prediction	
Observed Result	
Revision Reason	
Engineering Meaning	
Units Or Not Applicable	
Assumption	
Model Limit	
Next Test	
Help Trigger	
Minutes Used	

Notebook and submission procedure

1. Attempt, predict, run, inspect, and revise before opening a reveal.
2. Complete the end-of-notebook Independent Program Studio without a reveal or copied solution. Only those visibly labeled Studio cells are scored for independent mastery.
3. Save or download the completed notebook as one `.ipynb` file.
4. Upload that one notebook to the matching Gradescope assignment. Do not export a `.py` file or create a ZIP.